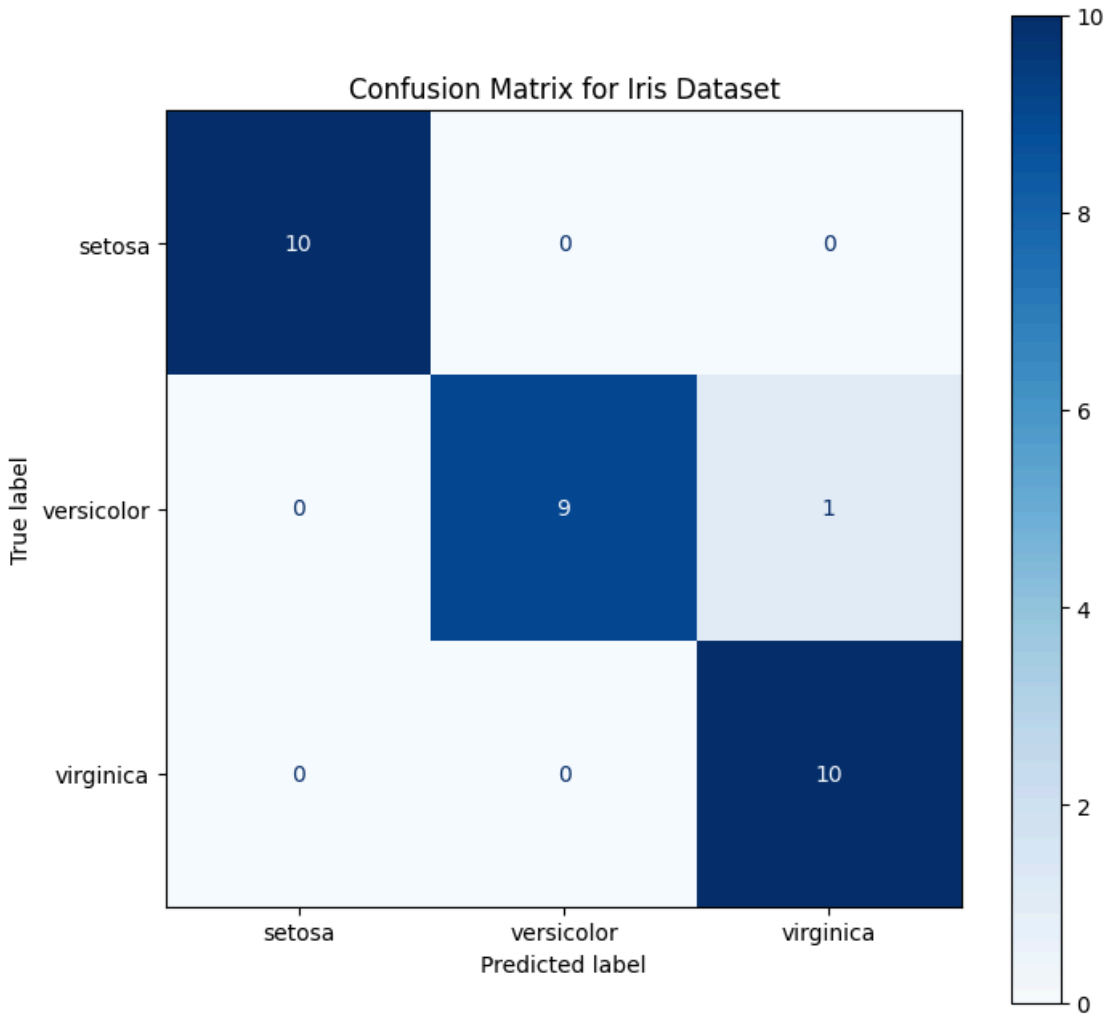
Practical 1 : Model Training And Evaluation

To train machine learning models using a given dataset and evaluate their performance using appropriate metrics, analyzing accuracy and effectiveness of the model.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
from sklearn.datasets import load_iris

# --- Data Loading and Initial Inspection ---
iris = load_iris()
X = iris.data # Features
y = iris.target # Target

# Create a DataFrame for better inspection
df = pd.DataFrame(X, columns=iris.feature_names)
df['species'] = iris.target_names[y]

# --- Split Data into Training and Testing Sets ---
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y)

# ---  Model Training ---
model = LogisticRegression(solver='liblinear', max_iter=200)

# Train the model using the training data
model.fit(X_train, y_train)

# ---- Model Evaluation ---
y_pred = model.predict(X_test)

# Evaluate the model using appropriate metrics for classification
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred, target_names=iris.target_names)

print(f"\nModel Evaluation on Test Set:")
print(f"Accuracy: {accuracy:.2f}")
print("\nClassification Report:")
print(report)

# You can also visualize the confusion matrix for more detailed evaluation
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(8, 8))
ConfusionMatrixDisplay.from_estimator(model, X_test, y_test,
display_labels=iris.target_names, cmap=plt.cm.Blues, ax=ax)
ax.set_title("Confusion Matrix for Iris Dataset")
plt.show()
```

```
Model Evaluation on Test Set:
Accuracy: 0.97

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      0.90      0.95        10
   virginica       0.91      1.00      0.95        10

    accuracy                           0.97        30
   macro avg       0.97      0.97      0.97        30
weighted avg       0.97      0.97      0.97        30
```



Practical 2 : Preprocessing

To perform data preprocessing techniques such as handling missing values, encoding categorical variables, and feature scaling to prepare raw data for machine learning models.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
from sklearn.impute import SimpleImputer, KNNImputer
import warnings
warnings.filterwarnings('ignore')

# --------------------- 1. Load Sample Dataset ---------------------
print("=== 1. Loading Dataset ===")
# Creating a sample dataset with common issues (missing values, outliers, categorical data)
data = {
    'Age': [25, 30, np.nan, 35, 40, 45, 50, 55, 60, np.nan, 28, 32],
    'Salary': [50000, 60000, 55000, np.nan, 80000, 90000, 120000, 150000,
               200000, 75000, 65000, 70000],
    'Gender': ['Male', 'Female', 'Male', 'Female', 'Male', np.nan, 'Female',
               'Male', 'Female', 'Male', 'Female', 'Male'],
    'City': ['Mumbai', 'Delhi', 'Bangalore', 'Mumbai', 'Delhi', 'Bangalore',
             'Mumbai', 'Delhi', 'Bangalore', 'Mumbai', 'Delhi', 'Bangalore'],
    'Purchased': ['No', 'Yes', 'No', 'Yes', 'Yes', 'No', 'Yes', 'Yes',
                  'No', 'Yes', 'No', 'Yes']
}

df = pd.DataFrame(data)
print("Original Dataset:")
print(df)
print("\nDataset Info:")
print(df.info())

# --------------------- 2. Handling Missing Values ---------------------
print("\n=== 2. Handling Missing Values ===")

# Check missing values
print("Missing Values:\n", df.isnull().sum())

# Method 1: Drop rows with missing values
df_drop = df.dropna()
print(f"\nAfter dropping missing rows: {df_drop.shape}")

# Method 2: Fill with Mean / Median / Mode
df_filled = df.copy()
df_filled['Age'] = df_filled['Age'].fillna(df_filled['Age'].mean())
df_filled['Salary'] = df_filled['Salary'].fillna(df_filled['Salary'].median())
df_filled['Gender'] = df_filled['Gender'].fillna(df_filled['Gender'].mode()[0])

print("\nAfter filling missing values:")
```

```
print(df_filled.isnull().sum())

# Method 3: Using SimpleImputer (Best Practice)
imputer_mean = SimpleImputer(strategy='mean')
imputer_median = SimpleImputer(strategy='median')
imputer_mode = SimpleImputer(strategy='most_frequent')

# --------------------- 3. Encoding Categorical Variables ---------------------
print("\n=== 3. Encoding Categorical Variables ===")

# Label Encoding (for ordinal or binary data)
le = LabelEncoder()
df_filled['Purchased_Encoded'] = le.fit_transform(df_filled['Purchased'])
print("Label Encoded 'Purchased':", df_filled['Purchased_Encoded'].unique())

# One-Hot Encoding (Best for nominal data)
onehot = OneHotEncoder(sparse_output=False, drop='first')
encoded_city = onehot.fit_transform(df_filled[['City']])
city_columns = onehot.get_feature_names_out(['City'])

df_encoded = pd.concat([df_filled, pd.DataFrame(encoded_city, columns=city_columns)],
axis=1)
print("\nAfter One-Hot Encoding:")
print(df_encoded.head())

# --------------------- 4. Feature Scaling / Normalization ---------------------
print("\n=== 4. Feature Scaling ===")

# Select numerical columns
numerical_cols = ['Age', 'Salary']

X = df_filled[numerical_cols]

# 1. Standard Scaling (Z-score) - Mean=0, Std=1
scaler_standard = StandardScaler()
X_standard = scaler_standard.fit_transform(X)
print("\nStandard Scaled (first 5 rows):")
print(pd.DataFrame(X_standard, columns=numerical_cols).head())

# 2. Min-Max Scaling (0 to 1)
scaler_minmax = MinMaxScaler()
X_minmax = scaler_minmax.fit_transform(X)
print("\nMin-Max Scaled (first 5 rows):")
print(pd.DataFrame(X_minmax, columns=numerical_cols).head())

# 3. Robust Scaling (Good for outliers)
scaler_robust = RobustScaler()
X_robust = scaler_robust.fit_transform(X)
print("\nRobust Scaled (first 5 rows):")
print(pd.DataFrame(X_robust, columns=numerical_cols).head())

# --------------------- 5. Handling Outliers ---------------------
print("\n=== 5. Handling Outliers ===")
```

```python
# Using IQR Method
Q1 = df_filled['Salary'].quantile(0.25)
Q3 = df_filled['Salary'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print(f"Salary - Lower Bound: {lower_bound}, Upper Bound: {upper_bound}")

# Capping outliers
df_no_outliers = df_filled.copy()
df_no_outliers['Salary'] = np.where(df_no_outliers['Salary'] > upper_bound, upper_bound,
df_no_outliers['Salary'])
df_no_outliers['Salary'] = np.where(df_no_outliers['Salary'] < lower_bound, lower_bound,
df_no_outliers['Salary'])

print("Shape before/after outlier handling:", df_filled.shape, "->", df_no_outliers.shape)

# --------------------- 6. Train-Test Split ---------------------
print("\n=== 6. Train-Test Split ===")

X_final = df_encoded.drop(['Purchased', 'City', 'Gender'], axis=1)  # Features
y_final = df_encoded['Purchased_Encoded']                          # Target

X_train, X_test, y_train, y_test = train_test_split(X_final, y_final,
                                                    test_size=0.2,
                                                    random_state=42)

print(f"Train shape: {X_train.shape}, Test shape: {X_test.shape}")

# --------------------- 7. Complete Preprocessing Pipeline Summary ---------------------
print("\n" + "="*60)
print("SUMMARY OF ALL BASIC PREPROCESSING OPERATIONS")
print("="*60)
print("1. Loading & Exploring Data")
print("2. Handling Missing Values (Drop / Mean / Median / Mode / Imputer)")
print("3. Encoding Categorical Data (Label + One-Hot Encoding)")
print("4. Feature Scaling (Standard, MinMax, Robust)")
print("5. Outlier Detection & Handling (IQR Method)")
print("6. Train-Test Split")
print("7. Feature Selection / Dropping unnecessary columns")
print("="*60)
```

```
=== 1. Loading Dataset ===
Original Dataset:
     Age    Salary  Gender       City Purchased
0   25.0   50000.0    Male     Mumbai        No
1   30.0   60000.0  Female      Delhi       Yes
2    NaN   55000.0    Male  Bangalore        No
3   35.0       NaN  Female     Mumbai       Yes
4   40.0   80000.0    Male      Delhi       Yes
5   45.0   90000.0     NaN  Bangalore        No
6   50.0  120000.0  Female     Mumbai       Yes
7   55.0  150000.0    Male      Delhi       Yes
8   60.0  200000.0  Female  Bangalore        No
9    NaN   75000.0    Male     Mumbai       Yes
10  28.0   65000.0  Female      Delhi        No
11  32.0   70000.0    Male  Bangalore       Yes

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 5 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Age        10 non-null     float64
 1   Salary     11 non-null     float64
 2   Gender     11 non-null     object
 3   City       12 non-null     object
 4   Purchased  12 non-null     object
dtypes: float64(2), object(3)
memory usage: 612.0+ bytes
None
```

```
=== 2. Handling Missing Values ===
Missing Values:
 Age          2
Salary       1
Gender       1
City         0
Purchased    0
dtype: int64

After dropping missing rows: (8, 5)

After filling missing values:
Age          0
Salary       0
Gender       0
City         0
Purchased    0
dtype: int64

=== 3. Encoding Categorical Variables ===
Label Encoded 'Purchased': [0 1]

After One-Hot Encoding:
    Age   Salary  Gender       City Purchased  Purchased_Encoded  City_Delhi  \
0  25.0  50000.0    Male     Mumbai        No                  0         0.0
1  30.0  60000.0  Female      Delhi       Yes                  1         1.0
2  40.0  55000.0    Male  Bangalore        No                  0         0.0
3  35.0  75000.0  Female     Mumbai       Yes                  1         0.0
4  40.0  80000.0    Male      Delhi       Yes                  1         1.0
```

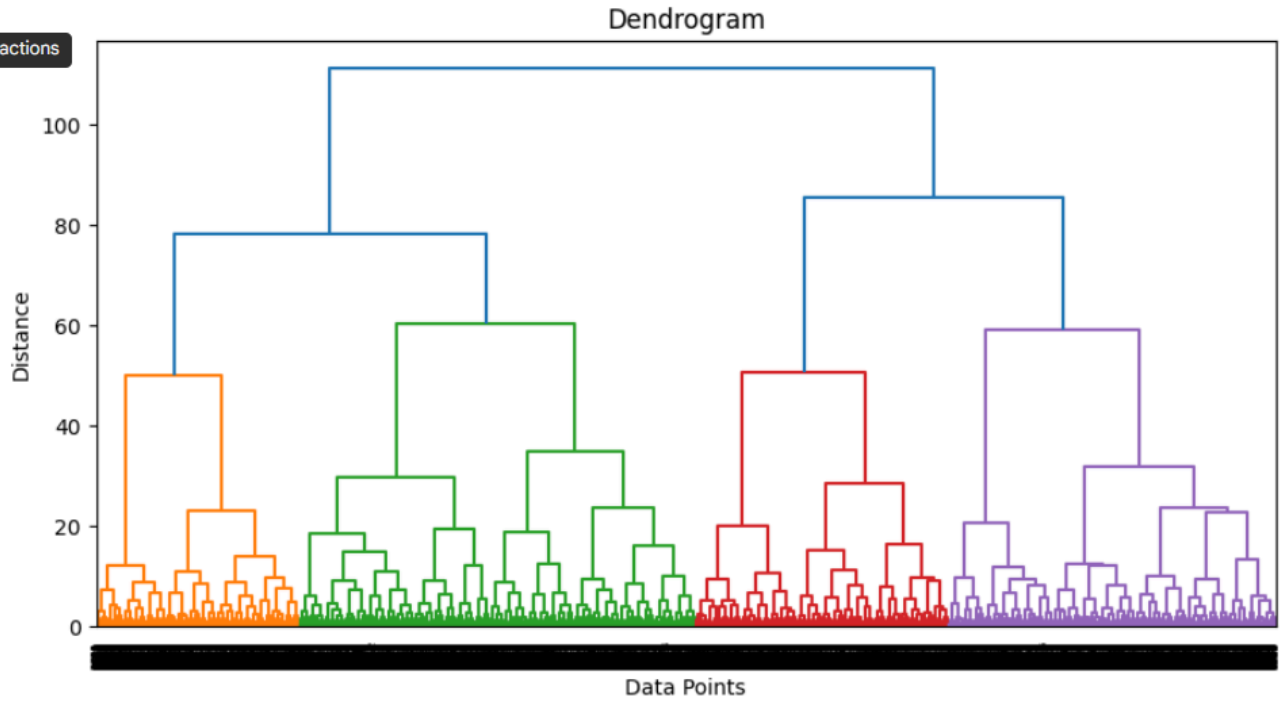
```
   City_Mumbai
0          1.0
1          0.0
2          0.0
3          1.0
4          0.0

=== 4. Feature Scaling ===

Standard Scaled (first 5 rows):
        Age    Salary
0 -1.436739 -0.956018
1 -0.957826 -0.721891
2  0.000000 -0.838955
3 -0.478913 -0.370701
4  0.000000 -0.253637

Min-Max Scaled (first 5 rows):
        Age    Salary
0  0.000000  0.000000
1  0.142857  0.066667
2  0.428571  0.033333
3  0.285714  0.166667
4  0.428571  0.200000

Robust Scaled (first 5 rows):
        Age    Salary
0 -1.016949 -0.740741
1 -0.677966 -0.444444
2  0.000000 -0.592593
3 -0.338983  0.000000
4  0.000000  0.148148

=== 5. Handling Outliers ===
Salary - Lower Bound: 13125.0, Upper Bound: 148125.0
Shape before/after outlier handling: (12, 6) -> (12, 6)

=== 6. Train-Test Split ===
Train shape: (9, 5), Test shape: (3, 5)

============================================================
SUMMARY OF ALL BASIC PREPROCESSING OPERATIONS
============================================================
1. Loading & Exploring Data
2. Handling Missing Values (Drop / Mean / Median / Mode / Imputer)
3. Encoding Categorical Data (Label + One-Hot Encoding)
4. Feature Scaling (Standard, MinMax, Robust)
5. Outlier Detection & Handling (IQR Method)
6. Train-Test Split
7. Feature Selection / Dropping unnecessary columns
============================================================
```

Practical 3 : Agglomerative Hierarchical Clustering

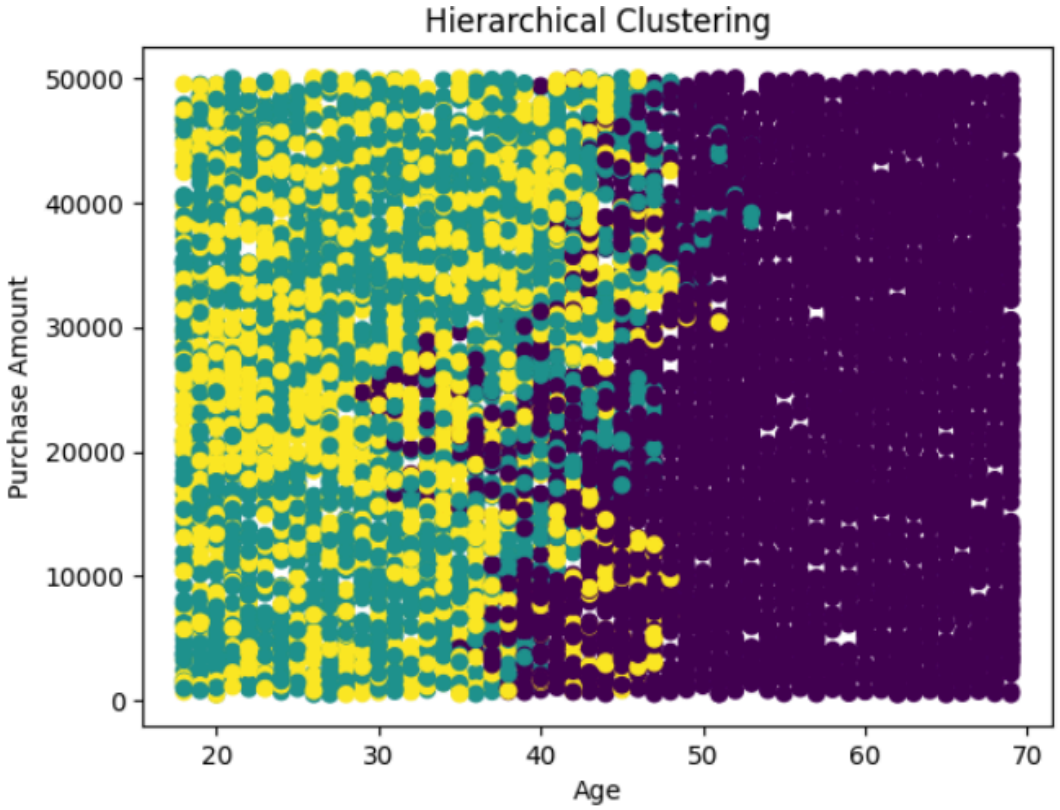
To implement Agglomerative Hierarchical Clustering on a dataset to group similar data points, visualize clusters using dendrograms, and analyze the hierarchical relationships among data.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage

df = pd.read_csv("customer_sales_9000.csv")

features = df[['Age', 'Purchase_Amount', 'Rating']]

scaler = StandardScaler()
scaled_data = scaler.fit_transform(features)

linked = linkage(scaled_data, method='ward')

plt.figure(figsize=(10,5))
dendrogram(linked)
plt.title("Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()

hc = AgglomerativeClustering(n_clusters=3, linkage='ward')
df['Cluster'] = hc.fit_predict(scaled_data)

print(df['Cluster'].value_counts())

plt.scatter(df['Age'], df['Purchase_Amount'], c=df['Cluster'])
plt.xlabel("Age")
plt.ylabel("Purchase Amount")
plt.title("Hierarchical Clustering")
plt.show()
```

output actions




```
Cluster
0    4576
1    2490
2    1934
Name: count, dtype: int64
```



Practical 4 : K-Nearest Neighbors (KNN)

This notebook demonstrates the K-Nearest Neighbour (KNN) classification algorithm using a synthetic dataset of job candidates. We will cover data preparation, model training with varying 'K' values, evaluation, and visualization of the decision boundary

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# Load the dataset
df_churn = pd.read_csv('/content/churn-bigml-80.csv')

# Separate target variable (Churn) from features
X = df_churn.drop('Churn', axis=1)
y = df_churn['Churn'].apply(lambda x: 1 if x == 'True.' else 0) # Convert
'True.'/'False.' to 1/0

# Identify categorical and numerical features
categorical_features = X.select_dtypes(include=['object', 'bool']).columns
numerical_features = X.select_dtypes(include=['int64', 'float64']).columns

# Create preprocessing pipelines for numerical and categorical features
numerical_transformer = StandardScaler()
categorical_transformer = OneHotEncoder(handle_unknown='ignore')

# Create a column transformer to apply different transformations to different columns
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_features),
        ('cat', categorical_transformer, categorical_features)
    ])

# Create the KNN model pipeline
# We'll use n_neighbors=5 as a starting point, but this can be tuned
knn_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', KNeighborsClassifier(n_neighbors=5))
])

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y)

# Train the KNN model
knn_pipeline.fit(X_train, y_train)

# Make predictions on the training set
```

```python
y_train_pred = knn_pipeline.predict(X_train)

# Make predictions on the test set
y_test_pred = knn_pipeline.predict(X_test)

# Evaluate the model on the training set
print("KNN Model Performance (Training Set):")
print(f"Accuracy: {accuracy_score(y_train, y_train_pred):.4f}")
print("\nClassification Report:")
print(classification_report(y_train, y_train_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_train, y_train_pred))

# Evaluate the model on the test set
print("\n" + "="*50)
print("KNN Model Performance (Test Set):")
print(f"Accuracy: {accuracy_score(y_test, y_test_pred):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_test_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_test_pred))
```

```
KNN Model Performance (Training Set):
Accuracy: 1.0000

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      2132

    accuracy                           1.00      2132
   macro avg       1.00      1.00      1.00      2132
weighted avg       1.00      1.00      1.00      2132


Confusion Matrix:
[[2132]]

==================================================
KNN Model Performance (Test Set):
Accuracy: 1.0000

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       534

    accuracy                           1.00       534
   macro avg       1.00      1.00      1.00       534
weighted avg       1.00      1.00      1.00       534


Confusion Matrix:
[[534]]
```

Practical 5 : Natural Language Processing (NLP)

To demonstrate fundamental Natural Language Processing (NLP) preprocessing techniques including tokenization, stopword removal, and stemming- and to convert raw text into numerical feature representations using Bag-of Words and TF-IDF vectorization, making text data suitable for machine learning models.

```
import nltk
import pandas as pd
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

documents = [
    "Data Science is an amazing field.",
    "Machine learning makes predictions easier.",
    "Text mining extracts useful information.",
    "Python is widely used for data analysis.",
    "NLP helps computers understand human language."
]

# Original documents (after lowercasing)
documents = [doc.lower() for doc in documents]
print("=== Lowercased Documents ===")
print(documents)

nltk.download('punkt')
nltk.download('punkt_tab') # Download punkt_tab as suggested by the error message

# Tokenized documents
tokenized_docs = [word_tokenize(doc) for doc in documents]
print("\n\n\n=== Tokenized Documents ===")
print(tokenized_docs)

nltk.download('stopwords')

# Filtered documents (stopwords removed)
stop_words = set(stopwords.words('english'))
filtered_docs = [[word for word in doc if word not in stop_words] for doc in
tokenized_docs]
print("\n\n\n=== Documents after Stopword Removal ===")
print(filtered_docs)

# Stemmed documents
stemmer = PorterStemmer()
stemmed_docs = [[stemmer.stem(word) for word in doc] for doc in filtered_docs]
print("\n\n\n=== Documents after Stemming ===")
print(stemmed_docs)

# Final processed documents (joined)
processed_docs = [" ".join(doc) for doc in stemmed_docs]
```

```python
print("\n\n\n=== Final Processed Documents ===")
print(processed_docs)

# Bag of Words (CountVectorizer) output
cv = CountVectorizer()
bow = cv.fit_transform(processed_docs)
print("\n\n\n=== Bag of Words (CountVectorizer)
==================================================================")
print(pd.DataFrame(bow.toarray(), columns=cv.get_feature_names_out()))

# TF-IDF Vectorizer output
tfidf = TfidfVectorizer()
tfidf_matrix = tfidf.fit_transform(processed_docs)
print("\n\n\n=== TF-IDF Vectorizer
==============================================================================")
print(pd.DataFrame(tfidf_matrix.toarray(), columns=tfidf.get_feature_names_out()))
```

```
=== Lowercased Documents ===
['data science is an amazing field.', 'machine learning makes predictions easier.', 'text mining extracts useful information.', 'python is widely used f


=== Tokenized Documents ===
[['data', 'science', 'is', 'an', 'amazing', 'field', '.'], ['machine', 'learning', 'makes', 'predictions', 'easier', '.'], ['text', 'mining', 'extracts'


=== Documents after Stopword Removal ===
[['data', 'science', 'amazing', 'field', '.'], ['machine', 'learning', 'makes', 'predictions', 'easier', '.'], ['text', 'mining', 'extracts', 'useful',


=== Documents after Stemming ===
[['data', 'scienc', 'amaz', 'field', '.'], ['machin', 'learn', 'make', 'predict', 'easier', '.'], ['text', 'mine', 'extract', 'use', 'inform', '.'], ['p


=== Final Processed Documents ===
['data scienc amaz field .', 'machin learn make predict easier .', 'text mine extract use inform .', 'python wide use data analysi .', 'nlp help comput
```

```
=== Bag of Words (CountVectorizer) ==============================================
   amaz  analysi  comput  data  easier  extract  field  help  human  inform  \
0     1        0       0     1       0        0      1     0      0       0
1     0        0       0     0       1        0      0     0      0       0
2     0        0       0     0       0        1      0     0      0       1
3     0        1       0     1       0        0      0     0      0       0
4     0        0       1     0       0        0      0     1      1       0

   ...  make  mine  nlp  predict  python  scienc  text  understand  use  wide
0  ...     0     0    0        0       0       1     0           0    0     0
1  ...     1     0    0        1       0       0     0           0    0     0
2  ...     0     1    0        0       0       0     1           0    1     0
3  ...     0     0    0        0       1       0     0           0    1     1
4  ...     0     0    1        0       0       0     0           1    0     0

[5 rows x 23 columns]
```

```
=== TF-IDF Vectorizer ==============================================================================
       amaz  analysi    comput      data    easier   extract     field  \
0  0.523358  0.00000  0.000000  0.422242  0.000000  0.000000  0.523358
1  0.000000  0.00000  0.000000  0.000000  0.447214  0.000000  0.000000
2  0.000000  0.00000  0.000000  0.000000  0.000000  0.463693  0.000000
3  0.000000  0.48214  0.000000  0.388988  0.000000  0.000000  0.000000
4  0.000000  0.00000  0.408248  0.000000  0.000000  0.000000  0.000000

       help     human    inform  ...      make      mine       nlp   predict  \
0  0.000000  0.000000  0.000000  ...  0.000000  0.000000  0.000000  0.000000
1  0.000000  0.000000  0.000000  ...  0.447214  0.000000  0.000000  0.447214
2  0.000000  0.000000  0.463693  ...  0.000000  0.463693  0.000000  0.000000
3  0.000000  0.000000  0.000000  ...  0.000000  0.000000  0.000000  0.000000
4  0.408248  0.408248  0.000000  ...  0.000000  0.000000  0.408248  0.000000

    python    scienc      text  understand       use     wide
0  0.00000  0.523358  0.000000    0.000000  0.000000  0.00000
1  0.00000  0.000000  0.000000    0.000000  0.000000  0.00000
2  0.00000  0.000000  0.463693    0.000000  0.374105  0.00000
3  0.48214  0.000000  0.000000    0.000000  0.388988  0.48214
4  0.00000  0.000000  0.000000    0.408248  0.000000  0.00000

[5 rows x 23 columns]
```

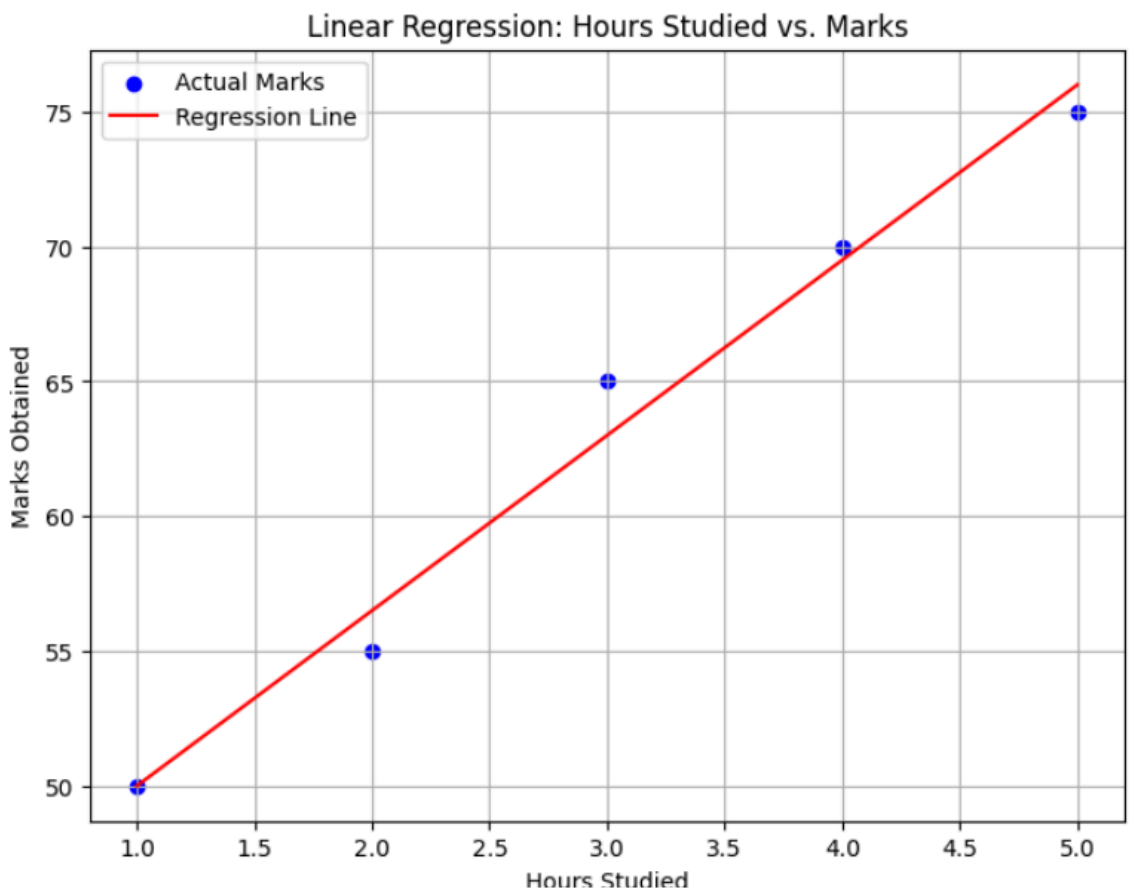
Practical 6 : Linear Regression

To build and evaluate a Simple Linear Regression model that predicts student exam marks based on study hours, demonstrating the relationship between a single independent variable and a continuous target variable using scikit-learn, and assessing model performance through MSE and R^2 metrics.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 1. Define the dataset for Linear Regression
# This dictionary contains 'Hours_Study', 'Sleep_Hours', and 'Marks' for students.
data = {
    'Hours_Study': [1, 2, 3, 4, 5],
    'Sleep_Hours': [7, 6, 7, 5, 6],
    'Marks': [50, 55, 65, 70, 75]
}

# Create a Pandas DataFrame from the data
df = pd.DataFrame(data)
print("=================== Original DataFrame ===================")
print(df)
print("\n") # Add a newline for better separation

# 2. Prepare the data for the model
# X is the feature (independent variable), which is 'Hours_Study'
X = df[['Hours_Study']]
# y is the target (dependent variable), which is 'Marks'
y = df['Marks']

# 3. Initialize and train the Linear Regression model
model = LinearRegression()
model.fit(X, y) # Fit the model to the training data

# 4. Make predictions using the trained model
y_pred = model.predict(X)

# 5. Evaluate the model's performance
# Mean Squared Error (MSE): Measures the average of the squares of the errors or
deviations.
# It quantifies the average magnitude of the errors.
print("=================== Model Evaluation ===================")
print(f"Mean Squared Error (MSE): {mean_squared_error(y, y_pred):.2f}")

# R2 Score: Represents the proportion of the variance in the dependent variable
# that is predictable from the independent variable(s). Closer to 1 is better.
print(f"R-squared (R2 Score): {r2_score(y, y_pred):.2f}")
print("\n")
```

```python
# 6. Visualize the results
plt.figure(figsize=(8, 6))
plt.scatter(X, y, color='blue', label='Actual Marks') # Plot actual data points
plt.plot(X, y_pred, color='red', label='Regression Line') # Plot the regression line
plt.xlabel("Hours Studied") # X-axis label
plt.ylabel("Marks Obtained") # Y-axis label
plt.title("Linear Regression: Hours Studied vs. Marks") # Plot title
plt.legend() # Display legend
plt.grid(True) # Add grid for better readability
plt.show() # Show the plot
```

```
=================== Original DataFrame ===================
   Hours_Study  Sleep_Hours  Marks
0            1            7     50
1            2            6     55
2            3            7     65
3            4            5     70
4            5            6     75

=================== Model Evaluation ===================
Mean Squared Error (MSE): 1.50
R-squared (R2 Score): 0.98
```



Practical 7 : Apriori Algorithm

To identify frequent itemsets and generate association rules from transaction data using the Apriori algorithm

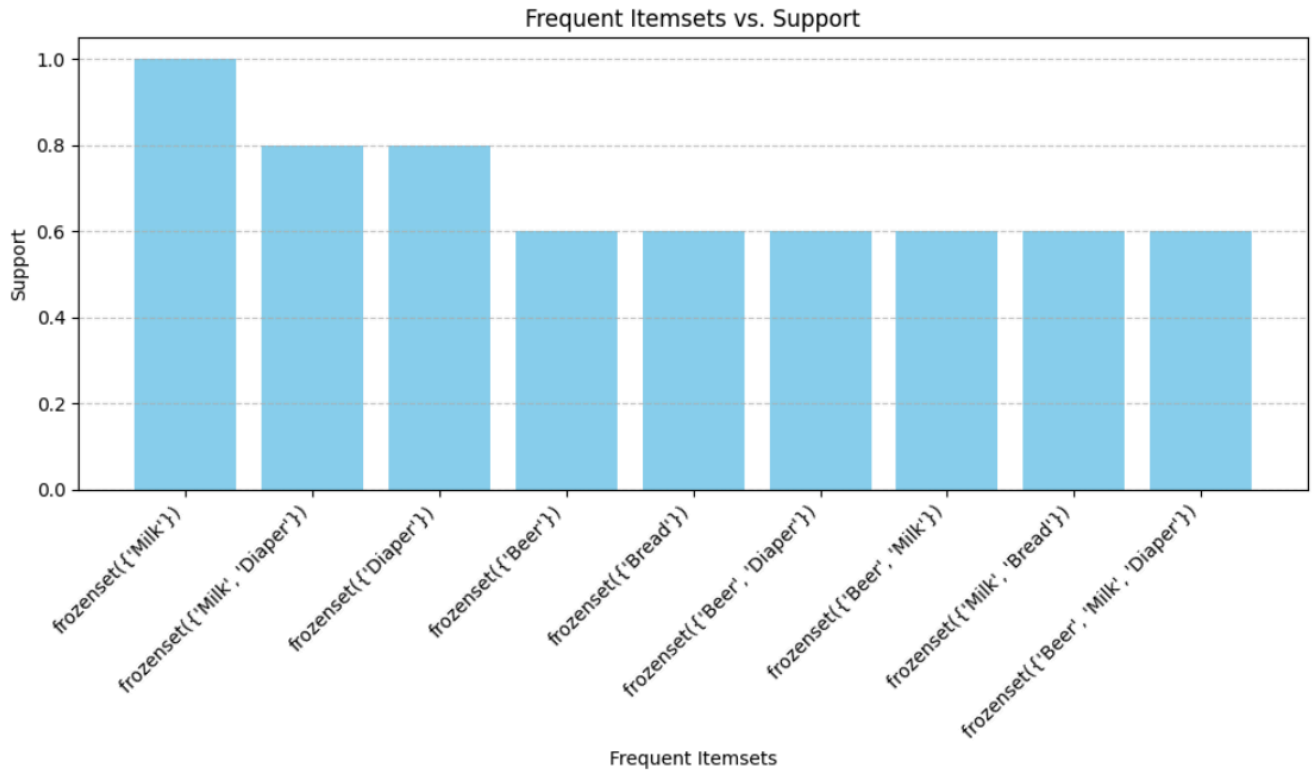
```
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder
import matplotlib.pyplot as plt
import networkx as nx
import warnings

# Suppress all warnings
warnings.filterwarnings('ignore')

# 1. Define the Transactional Dataset
# This dataset represents customer purchases, where each inner list is a transaction.
dataset = [
    ['Milk', 'Bread'],
    ['Milk', 'Diaper', 'Beer', 'Eggs'],
    ['Milk', 'Diaper', 'Beer', 'Coke'],
    ['Bread', 'Milk', 'Diaper', 'Beer'],
    ['Bread', 'Milk', 'Diaper', 'Coke']
]

# 2. Convert the transactional data into a one-hot encoded DataFrame
# TransactionEncoder transforms the list of transactions into a boolean DataFrame
# suitable for the Apriori algorithm.
te = TransactionEncoder()
te_data = te.fit(dataset).transform(dataset)
df = pd.DataFrame(te_data, columns=te.columns_)

print("=================== One-Hot Encoded Transactional Data ===================")
print(df)
print("\n") # Add a newline for better separation

# 3. Apply the Apriori algorithm to find frequent itemsets
# min_support=0.6 means an itemset must appear in at least 60% of transactions
# use_colnames=True displays item names instead of column indices.
frequent_items = apriori(df, min_support=0.6, use_colnames=True)

print("=================== Frequent Itemsets (min_support=0.6) ===================")
print(frequent_items)
print("\n")

# 4. Generate association rules from the frequent itemsets
# metric="confidence" and min_threshold=0.7 mean rules must have at least 70%
confidence.
# Confidence is the probability that a transaction contains Y given that it contains X.
rules = association_rules(frequent_items, metric="confidence", min_threshold=0.7)

print("=================== Generated Association Rules (min_confidence=0.7)
```

```
==================")
print(rules)
print("\n")

# 5. Visualize Frequent Itemsets
# Sort by support for better visualization
frequent_items.sort_values(by='support', ascending=False, inplace=True)

# Convert itemsets to string for plotting as bar chart labels
frequent_items['itemsets'] = frequent_items['itemsets'].astype(str)

plt.figure(figsize=(10, 6))
plt.bar(frequent_items['itemsets'], frequent_items['support'], color='skyblue')
plt.xticks(rotation=45, ha='right') # Rotate labels for better readability
plt.xlabel("Frequent Itemsets")
plt.ylabel("Support")
plt.title("Frequent Itemsets vs. Support")
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

# 6. Visualize Association Rules as a Network Graph
# This helps to understand the relationships and strength (lift) between rules.
G = nx.DiGraph()

for _, row in rules.iterrows():
    antecedent_str = ', '.join(list(row['antecedents']))
    consequent_str = ', '.join(list(row['consequents']))
    G.add_edge(frozenset(row['antecedents']), frozenset(row['consequents']),
               weight=row['lift'], label=f"Lift: {row['lift']:.2f}")

plt.figure(figsize=(12, 8))
pos = nx.spring_layout(G, k=0.5, iterations=50) # Layout for node positioning

nx.draw_networkx_nodes(G, pos, node_size=3000, node_color='lightgreen', alpha=0.9)
nx.draw_networkx_edges(G, pos, edge_color='gray', width=1.5, alpha=0.6, arrowsize=20)
nx.draw_networkx_labels(G, pos, font_size=10, font_weight='bold')

edge_labels = nx.get_edge_attributes(G, 'label')
nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, font_color='red',
font_size=8)

plt.title("Association Rules Network Graph")
plt.axis('off') # Hide axes
plt.tight_layout()
plt.show()
```

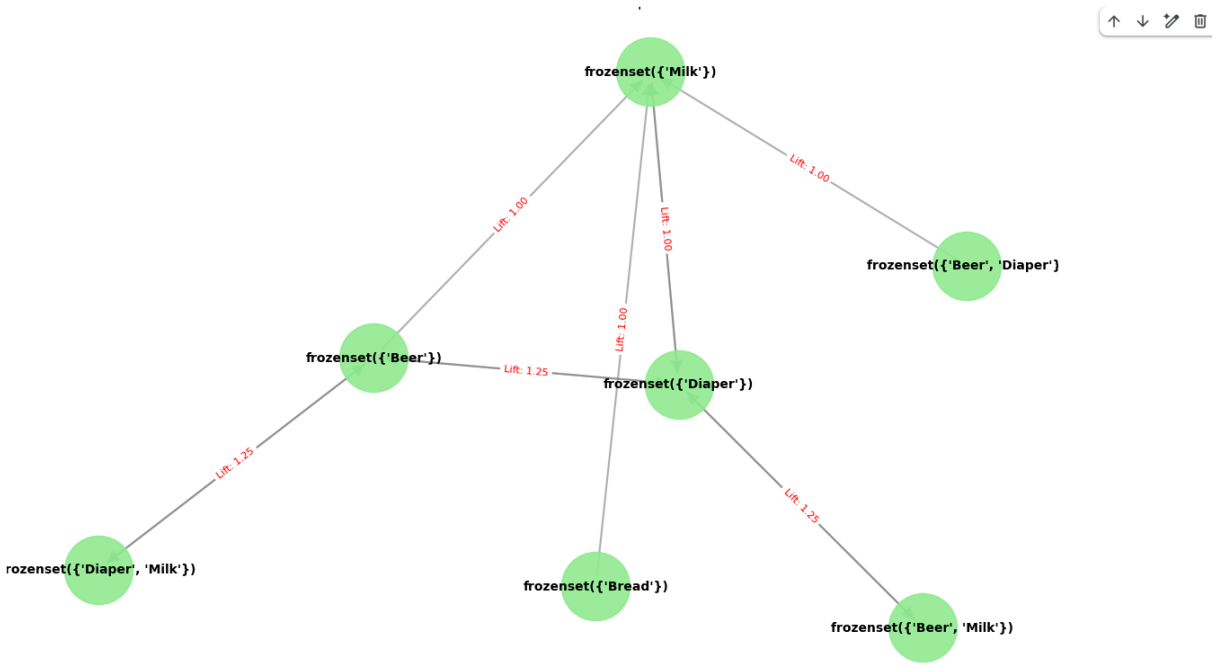
```
  return datetime.utcnow().replace(tzinfo=utc)
=================== One-Hot Encoded Transactional Data ===================
    Beer  Bread   Coke  Diaper   Eggs  Milk
0  False   True  False   False  False  True
1   True  False  False    True   True  True
2   True  False   True    True  False  True
3   True   True  False    True  False  True
4  False   True   True    True  False  True


=================== Frequent Itemsets (min_support=0.6) ===================
   support              itemsets
0      0.6                 (Beer)
1      0.6                (Bread)
2      0.8               (Diaper)
3      1.0                 (Milk)
4      0.6         (Beer, Diaper)
5      0.6           (Beer, Milk)
6      0.6          (Milk, Bread)
7      0.8         (Milk, Diaper)
8      0.6  (Beer, Milk, Diaper)


=================== Generated Association Rules (min_confidence=0.7) ===================
       antecedents     consequents  antecedent support  consequent support  \
0           (Beer)        (Diaper)                 0.6                 0.8
1         (Diaper)          (Beer)                 0.8                 0.6
2           (Beer)          (Milk)                 0.6                 1.0
3          (Bread)          (Milk)                 0.6                 1.0
4           (Milk)        (Diaper)                 1.0                 0.8
5         (Diaper)          (Milk)                 0.8                 1.0
6     (Beer, Milk)        (Diaper)                 0.6                 0.8
7   (Beer, Diaper)          (Milk)                 0.6                 1.0
8   (Diaper, Milk)          (Beer)                 0.8                 0.6
9           (Beer)  (Diaper, Milk)                 0.6                 0.8
10        (Diaper)    (Beer, Milk)                 0.8                 0.6

    support  confidence  lift  representativity  leverage  conviction  \
0       0.6        1.00  1.25               1.0      0.12         inf
1       0.6        0.75  1.25               1.0      0.12         1.6
2       0.6        1.00  1.00               1.0      0.00         inf
3       0.6        1.00  1.00               1.0      0.00         inf
4       0.8        0.80  1.00               1.0      0.00         1.0
5       0.8        1.00  1.00               1.0      0.00         inf
6       0.6        1.00  1.25               1.0      0.12         inf
7       0.6        1.00  1.00               1.0      0.00         inf
8       0.6        0.75  1.25               1.0      0.12         1.6
9       0.6        1.00  1.25               1.0      0.12         inf
10      0.6        0.75  1.25               1.0      0.12         1.6
```

	zhangs_metric	jaccard	certainty	kulczynski
0	0.5	0.75	1.000	0.875
1	1.0	0.75	0.375	0.875
2	0.0	0.60	0.000	0.800
3	0.0	0.60	0.000	0.800
4	0.0	0.80	0.000	0.900
5	0.0	0.80	0.000	0.900
6	0.5	0.75	1.000	0.875
7	0.0	0.60	0.000	0.800
8	1.0	0.75	0.375	0.875
9	0.5	0.75	1.000	0.875
10	1.0	0.75	0.375	0.875

...



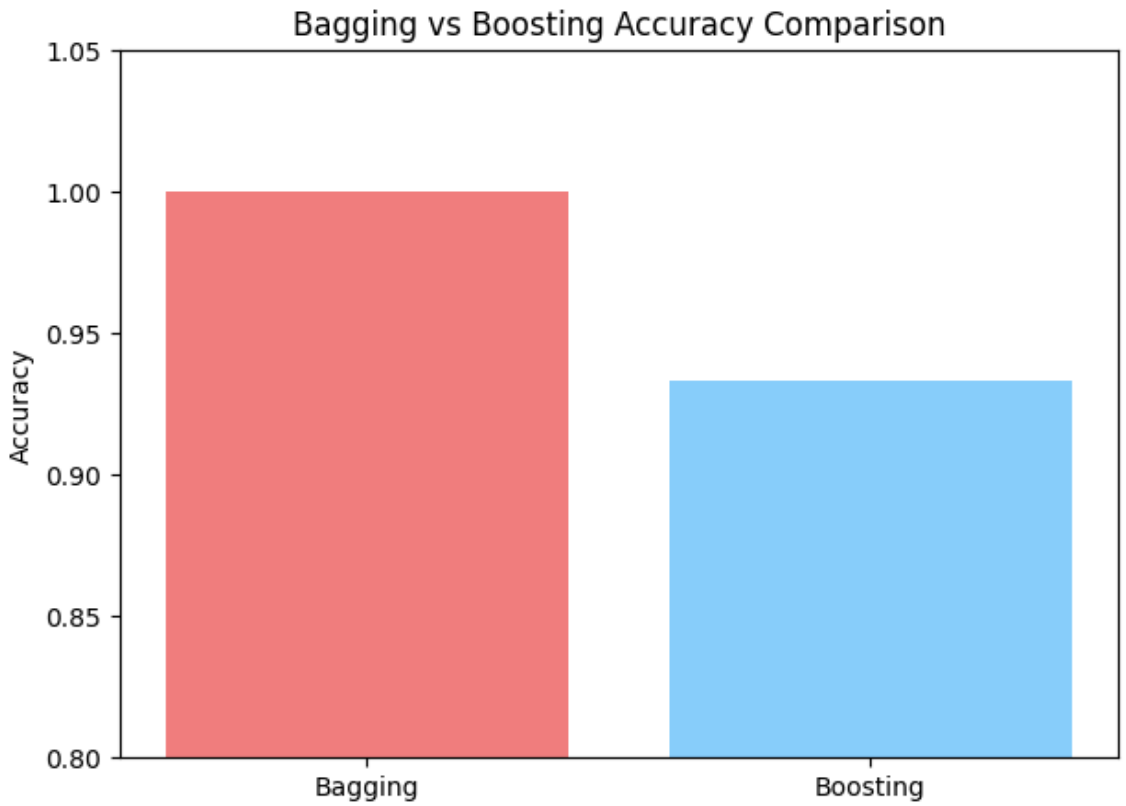
...



Practical 8 : Bagging And Boosting

Bagging and Boosting. To implement and compare two ensemble learning techniques - Bagging and Boosting on the Iris dataset, evaluating how each method improves classification performance over a single model

```
from sklearn.ensemble import BaggingClassifier, AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# 1. Load the dataset
# The Iris dataset is a classic and simple dataset for classification.
data = load_iris()
X = data.data # Features (sepal length, sepal width, petal length, petal width)
y = data.target # Target variable (species of iris: setosa, versicolor, virginica)

# 2. Split the dataset into training and testing sets
# We use 80% of the data for training and 20% for testing to evaluate generalization.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# ==================== Bagging Model ====================
# 3. Initialize and train a Bagging Classifier
# Bagging (Bootstrap Aggregating) builds multiple models (here, Decision Trees)
# on different bootstrap samples of the training data and averages their predictions.
bag_model = BaggingClassifier(
    estimator=DecisionTreeClassifier(), # Base estimator: a Decision Tree
    n_estimators=10, # Number of base estimators (trees) to build
    random_state=42 # For reproducibility
)
bag_model.fit(X_train, y_train) # Train the Bagging model

# 4. Make predictions using the Bagging model
y_pred_bag = bag_model.predict(X_test)

# 5. Evaluate the Bagging model
print("==================== Bagging Model Performance ====================")
print(f"Bagging Accuracy: {accuracy_score(y_test, y_pred_bag):.4f}") # Print accuracy,
formatted to 4 decimal places
print("\n")

# ==================== Boosting Model (AdaBoost) ====================
# 6. Initialize and train an AdaBoost Classifier
# AdaBoost (Adaptive Boosting) trains weak learners sequentially, with each
# subsequent learner focusing on samples misclassified by previous ones.
boost_model = AdaBoostClassifier(
    n_estimators=50, # Number of weak learners (stumps by default)
    random_state=42 # For reproducibility
)
```

```
boost_model.fit(X_train, y_train) # Train the AdaBoost model

# 7. Make predictions using the Boosting model
y_pred_boost = boost_model.predict(X_test)

# 8. Evaluate the Boosting model
print("=================== Boosting Model Performance ===================")
print(f"Boosting Accuracy: {accuracy_score(y_test, y_pred_boost):.4f}") # Print
accuracy, formatted to 4 decimal places
print("\n")

# ==================== Visualization ====================
# 9. Compare Bagging and Boosting accuracies visually
models = ['Bagging', 'Boosting']
accuracy = [
    accuracy_score(y_test, y_pred_bag),
    accuracy_score(y_test, y_pred_boost)
]

plt.figure(figsize=(7, 5))
plt.bar(models, accuracy, color=['lightcoral', 'lightskyblue'])
plt.ylim(0.8, 1.05) # Set y-axis limits for better comparison
plt.ylabel("Accuracy")
plt.title("Bagging vs Boosting Accuracy Comparison")
plt.show()
```

```
=================== Bagging Model Performance ===================
Bagging Accuracy: 1.0000


=================== Boosting Model Performance ===================
Boosting Accuracy: 0.9333
```